Dodge Blitz: Accelerometer-Controlled VGA Dodge Game on Nexys A7 FPGA

Ardaan Bhatia

Kyna Rochlani

EE 354L: Introduction to Digital Logic

Spring 2026

I. Abstract

Dodge Blitz is a real-time falling-block arcade game implemented entirely in synthesizable Verilog on a Digilent Nexys A7 FPGA board (Artix-7). The system outputs 640×480 VGA video, reads board-tilt input from the onboard ADXL362 accelerometer over SPI, and displays the score on the seven-segment display. Players choose one of three visual themes, then maneuver a 40×40 pixel sprite to avoid red obstacle blocks that fall with increasing speed. The game demonstrates integration of VGA timing generation, SPI communication, hardware pseudo-randomness, sprite ROM rendering, and a synchronous three-state FSM within a single FPGA design.

II. Introduction and Background

This project builds directly on techniques developed in EE 354L lab exercises: VGA timing generation from lab experiments with hSync/vSync counters, seven-segment display multiplexing from earlier homework, and synchronous FSM design practiced throughout the semester. The ADXL362 SPI accelerometer interface extends the SPI communication concepts covered in lecture. Prior work on pixel-coordinate-based drawing established the foundation for the procedural background rendering used here. Together, these components are combined into a complete interactive system that exercises the full hardware design flow from RTL to bitstream on a real FPGA board.

III. The Design

A. Design Description. Dodge Blitz is a single-player dodge game rendered over VGA at 640×480 60 Hz. After power-on, the player selects one of three themes, Space, Blue Sky, or Garden, each with a unique procedurally drawn background and a 40×40 pixel sprite stored in a Verilog ROM (spaceship, bird, or dog). During gameplay, red 20×20 pixel blocks fall from the top of the screen at increasing speed. The player moves their sprite left or right by physically tilting the Nexys A7 board; the onboard ADXL362 accelerometer provides the tilt signal via SPI. The score increments each game tick and is shown on the seven-segment display. If a falling block overlaps the player sprite, one life is lost (three lives total). When all lives are gone, a game-over screen displays the final score. The project is organized into six Verilog source files:

A7_vga_top.v: top module

display_controller.v: VGA + game FSM

accelerometer_reader.v: SPI master

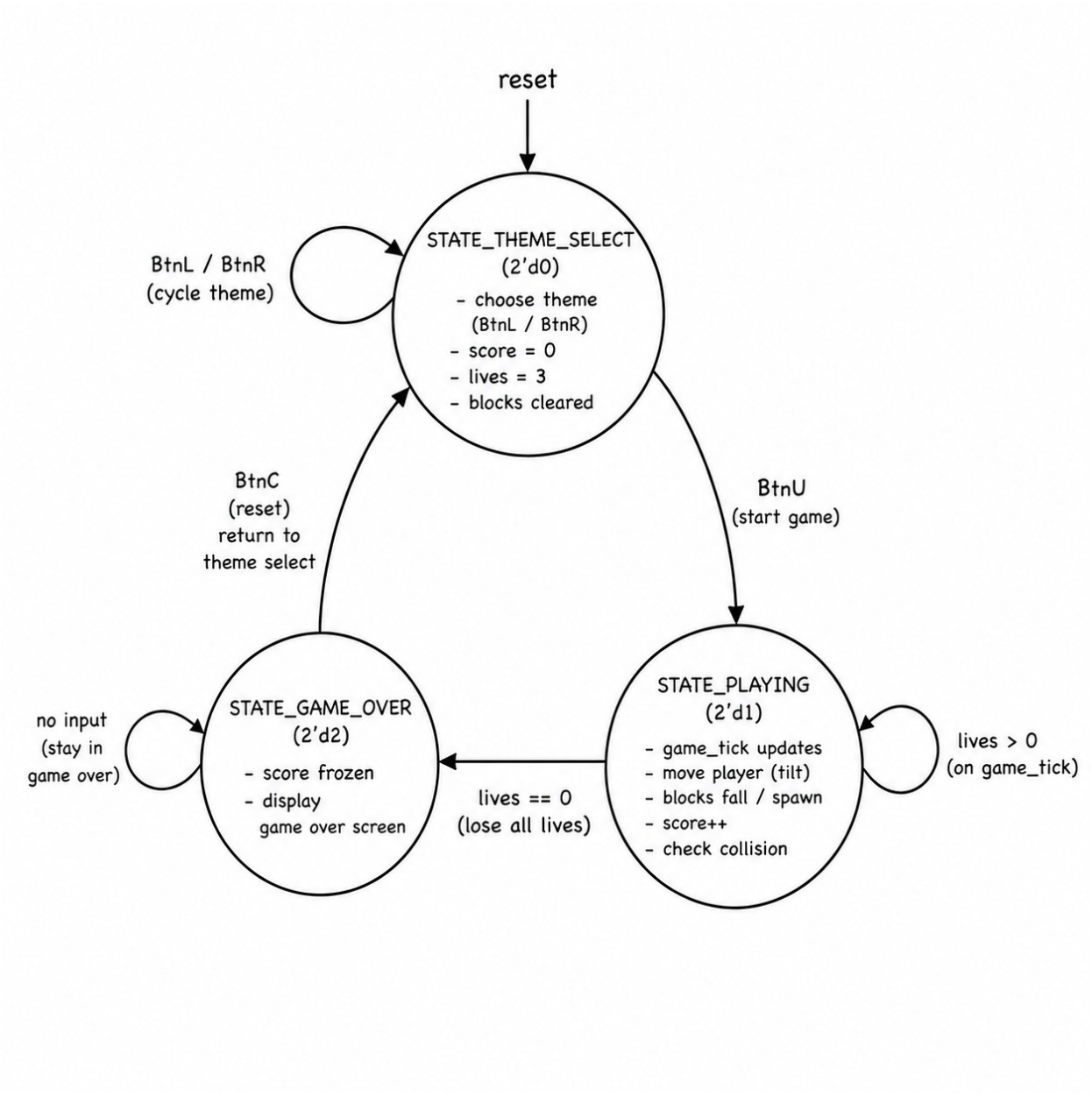
lfsr.v: 8-bit pseudo-random generator

counter.v: seven-segment driver

Three sprite ROM files (12-bit, 40×40 px)

B. State Machine. The primary FSM resides in display_controller.v and uses a 2-bit register with three states encoded as localparams. All state updates are synchronous (posedge clk).

State	Encoding	Behavior & Transitions
`STATE_THEME_SELECT`	`2'd0`	Initial state after reset. BtnL/BtnR cycle theme; BtnU → STATE_PLAYING. All game variables reset (lives=3, score=0, blocks cleared).
`STATE_PLAYING`	`2'd1`	Game updates each game_tick. Accelerometer moves player; blocks fall and spawn via LFSR X-positions; score increments; AABB collision decrements lives. lives==0 → STATE_GAME_OVER.
`STATE_GAME_OVER`	`2'd2`	Score frozen; game-over panel rendered over theme background. BtnC → STATE_THEME_SELECT (global reset).



The game clock-enable signal game_tick is derived by counting 25 MHz pixel clock cycles and pulsing for one cycle at the desired gameplay rate. Gating all state updates on game_tick decouples visible motion speed from the pixel clock without introducing clock domain crossings. The accelerometer_reader SPI FSM cycles through states ST_START_WRITE → ST_START_READ → ST_SHIFT_LOW → ST_SHIFT_HIGH → ST_FINISH → ST_WAIT, reading the signed 12-bit Y-axis register. Values below −80 assert tilt_left; values above +80 assert tilt_right; values within the dead zone assert tilt_neutral. An 8-bit LFSR (lfsr.v) advances every clock cycle and is sampled at block spawn events to generate varied obstacle X positions. Collision detection uses axis-aligned bounding box (AABB) overlap: a collision is flagged when player_x < block_x[i]+20 AND player_x+40 > block_x[i] AND PLAYER_Y < block_y[i]+20 AND PLAYER_Y+40 > block_y[i]. Block fall speed scales with score in seven tiers from 2 px/tick (score 0–9) up to 8 px/tick (score 60+).

C. User Interface. Inputs and outputs are summarized in the table below. The VGA output renders all graphics at 640×480 60 Hz using a 25 MHz pixel clock derived inside the display controller. Three LEDs (LED[0/1/2]) mirror tilt_left, tilt_right, and tilt_neutral for hardware debugging. The seven-segment display shows the live score during gameplay and the final score on game over.

Input	Active in	Function
BtnC	All states	Global reset to theme select
BtnU	Theme select	Confirm theme, start game
BtnL / BtnR	Theme select	Cycle theme left / right
Tilt left	Gameplay	Move player sprite left
Tilt right	Gameplay	Move player sprite right
Flat / neutral	Gameplay	Player holds position

The rendering pipeline resolves pixel color in priority order: (1) UI elements (hearts, score label, flash message), (2) red obstacle blocks, (3) player sprite with magenta (12'hF0F) transparency, (4) theme background. All background graphics are procedural, where Space uses a near-black field with star dots and a planet ellipse, Blue Sky uses a light-blue fill with cloud ellipses and a sun circle, Garden uses a bicolor sky/lawn split with rectangular trees and flower pixels. Because sprite ROMs have one-cycle registered output latency, address computation is advanced one pipeline stage so pixel data arrives aligned with the correct screen coordinates.

IV. Test Methodology

Testing was performed iteratively on hardware following each development milestone. No formal testbench simulation was used; verification relied on direct observation of VGA output, seven-segment display readings, and the tilt debug LEDs.

Reset: BtnC was pressed from all three game states to confirm unconditional return to STATE_THEME_SELECT with score cleared to 0, lives restored to 3, player centered, and all block slots deactivated.

Theme selection: BtnL and BtnR were cycled through all three themes in both directions. BtnU was pressed from each theme to confirm the correct sprite and background appeared during gameplay.

Accelerometer: The board was tilted left, right, and held flat. LED[0], LED[1], and LED[2] confirmed tilt_left, tilt_right, and tilt_neutral signals respectively. The dead zone was verified by gently rocking near horizontal and confirming no player movement below the ±80 threshold. Player boundary clamping was tested by tilting fully to each edge.

Collision and lives: Intentional collisions were induced by positioning the player beneath descending blocks. Each collision was confirmed to deactivate the block, decrement the heart count, and display the "YOU LOST LIFE"

flash. The sequence was repeated to reduce lives from 3 to 0, and the FSM transition to STATE_GAME_OVER was verified on the final collision.

Score and difficulty: The seven-segment display was observed to increment at a regular rate. Block fall speed was confirmed to increase visibly at each 10-point tier boundary up to the 60+ maximum. Score freeze on game over was verified. Block spawn X-position variation across LFSR cycles was confirmed visually across multiple gameplay sessions.

Game-over screen: The final score in the panel was confirmed to match the seven-segment reading at the moment of the last life lost. The blinking GAME OVER title, bordered panel, dark overlay, and underlying background were all verified to render without overlap or pixel artifacts after layout adjustment.

V. Conclusion and Future Work

Dodge Blitz successfully demonstrates a complete interactive game implemented in synthesizable Verilog with no processor, OS, or external memory. The design integrates VGA generation, SPI accelerometer communication, LFSR pseudo-randomness, sprite ROMs with transparency, layered pixel rendering, and a structured FSM into a single cohesive system. The most important lessons learned were: separating game update rate from the pixel clock early in the design (using game_tick), compensating for sprite ROM pipeline latency explicitly, and using on-board debug LEDs to verify subsystem behavior without simulation. Iterative hardware testing on the real VGA monitor was essential for catching rendering layout issues that would be difficult to catch in simulation.

Several extensions are well-suited for a future EE 354L project. Adding a PWM-based buzzer module for collision and game-over sound effects would be a natural next step requiring only a counter and a pin assignment. Storing the all-time high score in a small block RAM would introduce BRAM inference. Rendering the numeric score directly on-screen using a pixel-font ROM would combine ROM design with VGA rendering skills from existing labs. Adding a pause state would exercise button-triggered FSM transitions. More ambitious extensions include animated obstacles or an accelerometer-calibrated difficulty option on the theme selection screen.

Regarding course feedback: the progression from combinational logic through sequential design to the FPGA project provided a coherent path. Having a complete VHDL/Verilog reference for common patterns (debouncing, SPI, VGA timing) available earlier in the semester would reduce startup time on the final project. The VGA and SPI labs were among the most engaging, as they produced immediately visible and tangible results. Future students would benefit from a short guided lab on the ADXL362 SPI protocol before the open-ended project phase, since debugging a custom SPI master from scratch against a datasheet is one of the more time-consuming aspects of accelerometer-based projects.